FACULTAD DE INGENIERÍA

CARRERA PROFESIONAL DE INGENIERÍA DE SOFTWARE

LABORATORIO

Curso: JAVASCRIPT AVANZADO

- **Rosas Ortiz Exon Gabriel**
- **Lopez Flores Angel David**

23 de febrero de 2026.

Introducción 3
Descripción de la organización 3
Proceso AS IS 4
Proceso TO BE 5
Desarrollo del Sistema – Backend (Node.js) 6
Archivos código origen: 7
Archivos raíz: 7
Beneficios de la arquitectura modular: 8
Librería principal utilizada: 8
Desarrollo del Frontend – Versión Inicial 8
Migración y Mejora con Angular 11
Desarrollo del Landing Page – React 20
Conclusiones: 24
Cuadro comparativo 25
Bibliografia: 26

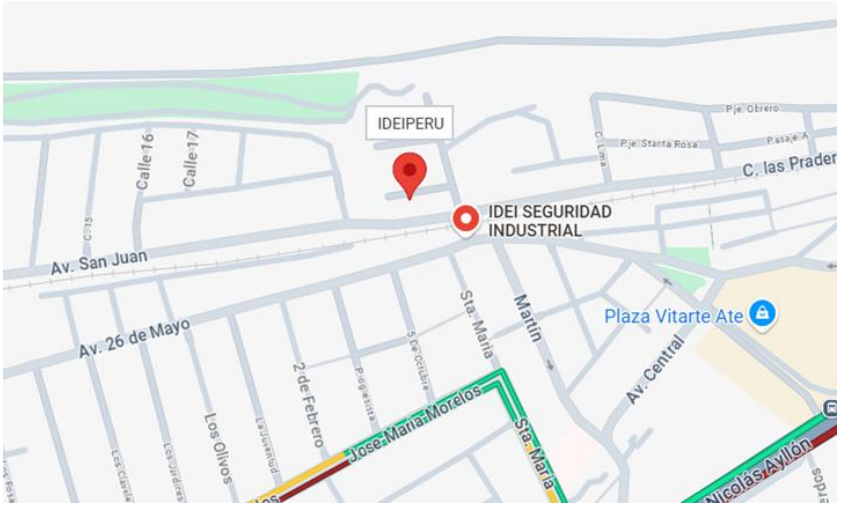
Introducción

Descripción de la organización

IDEI es una empresa peruana con más de 11 años de experiencia en la creación de soluciones electrónicas innovadoras para diversos sectores, como la seguridad, control y automatización, optimización de energía, mantenimiento de equipos y maquinarias, e iluminación, diseñadas para satisfacer las necesidades específicas de nuestros clientes. Comprometidos con brindar un servicio personalizado y eficaz, respaldado por un personal altamente capacitado. IDEI ofrece una amplia gama de luminarias, sistemas de monitoreo, y productos de señalización, consolidándose como un referente en la provisión de soluciones tecnológicas especializadas para un sector industrial que requiere alta eficiencia y seguridad.

Figura 1

Ubicación Geográfica de IDEI S.A.C



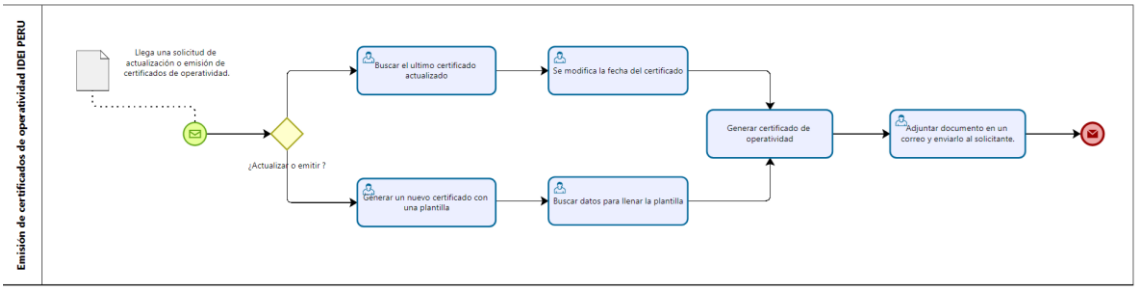
Psj. Rosa Manuel Letra E Urb. Santa Rosa, Ate Vitarte - Lima, PERÚ. Referencia Altura del Grifo Tokio de la AV. Carretera central. ATE

Proceso AS IS

Representación visual y detallada de cómo funciona un proceso actualmente en una organización, identificando cada paso, participante, herramienta y flujo de información para entender su estado real, incluyendo errores, cuellos de botella e ineficiencias, sirviendo como base para compararlo con un futuro "To Be" (cómo debería ser) y poder mejorarlo.

Figura 2

Proceso de emisión de certificados de operatividad de IDEI S.A.C



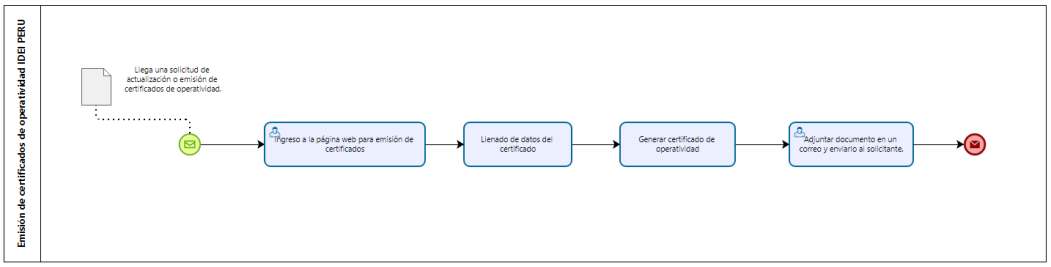
Representación de las actividades, decisiones y actores involucrados desde la solicitud del servicio hasta la entrega del certificado final.

Proceso TO BE

Representación gráfica del futuro deseado de un proceso de negocio, diseñado para ser más eficiente, alineado con la estrategia y optimizado tras identificar fallos en el estado actual ("As-Is"). Se enfoca en la mejora, la automatización, la eliminación de desperdicios y la incorporación de nuevas tecnologías para alcanzar objetivos organizacionales, definiendo nuevos roles, pasos y normas.

Figura 3

Proceso de emisión de certificados de operatividad optimizado de IDEI S.A.C



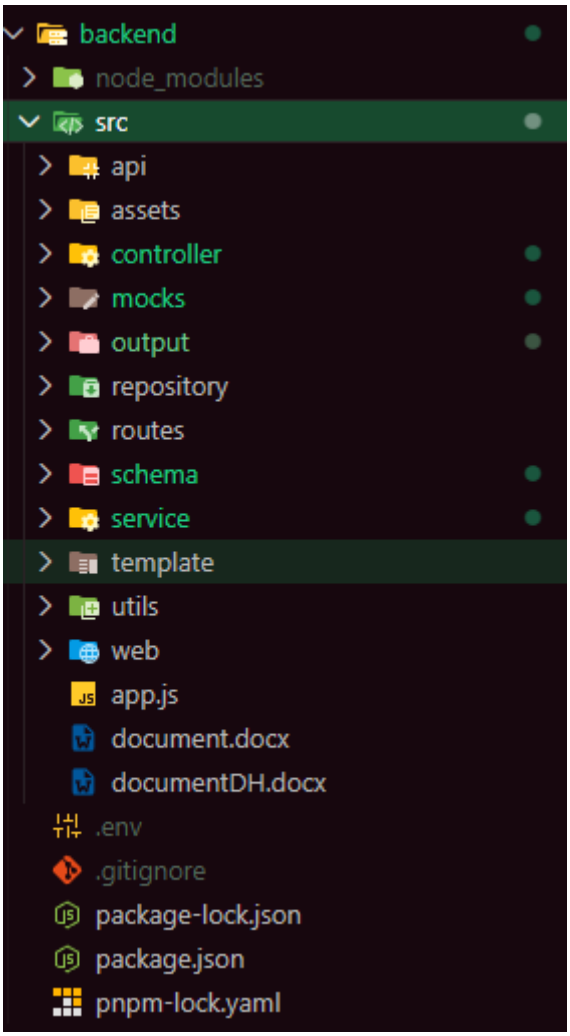
Representación de las actividades, decisiones y actores involucrados desde la solicitud del servicio hasta la entrega del certificado final.

Desarrollo del Sistema – Backend (Node.js)

Node.js es un entorno de ejecución JavaScript asíncrono basado en eventos, Node.js está diseñado para construir aplicaciones de red escalables.

Figura 4

Arquitectura modular Backend.



El proyecto se organiza bajo un enfoque modular, lo que permite separar responsabilidades, mejorar la mantenibilidad y facilitar la escalabilidad del sistema.

Archivos código origen:

- **src/:** Contiene el núcleo de la aplicación, dividido en submódulos especializados:
    - api/: Interfaces de comunicación y definición de endpoints.
    - controller/: Lógica de control que gestiona las solicitudes y coordina la interacción entre servicios y repositorios.
    - repository/: Capa de acceso a datos, encargada de la persistencia y consultas a la base de datos.
    - service/: Implementa la lógica de negocio y reglas de validación.
    - routes/: Configuración de rutas y vinculación con los controladores.
    - schema/: Definición de modelos y validaciones de datos.
    - assets/, mocks/, output/, template/: Soporte adicional para pruebas, recursos estáticos, generación de resultados y plantillas.
- **utils/:** Funciones auxiliares reutilizables que apoyan la lógica principal.
- **web/:** Módulo destinado a la integración con la capa web o frontend.

Archivos raíz:

- **app.js:** Punto de entrada de la aplicación, encargado de inicializar la configuración y levantar el servidor.
- **.env:** Variables de entorno para la configuración segura.
- **package.json, package-lock.json, pnpm-lock.yaml:** Gestión de dependencias y control de versiones.
- **.gitignore:** Exclusiones para el control de versiones.

Beneficios de la arquitectura modular:

Separación de capas: Controladores, servicios y repositorios están claramente diferenciados, siguiendo principios de arquitectura limpia.

Escalabilidad: La modularidad permite añadir nuevas funcionalidades sin afectar el núcleo existente.

Mantenibilidad: La organización facilita la localización de errores y la extensión del código.

Buenas prácticas: Uso de esquemas para validación, utilidades centralizadas y configuración externa mediante ".env".

Librería principal utilizada:

Puppeteer: Para la generación de PDF a partir de plantillas HTML, inicialmente se intentó convertir archivos .docx a .pdf, pero debido a la complejidad técnica y la necesidad de herramientas de pago, se optó por diseñar una plantilla HTML-CSS que replica el certificado original casi al 100%.

CORS: Mmiddleware esencial de Express que habilita el Intercambio de Recursos de Origen Cruzado (CORS), permitiendo que un servidor web (backend) acepte solicitudes de una aplicación frontend alojada en un dominio, puerto o protocolo diferente, solucionando errores de bloqueo del navegador.

Express: Framework web para Node. Js diseñado para crear aplicaciones web y APIs robustas de forma rápida, minimalista y flexible. Proporciona una capa ligera de funcionalidades.

Dotenv: Herramienta fundamental para gestionar la configuración de aplicaciones, permitiendo cargar variables de entorno desde un archivo .env al entorno de ejecución (process.env en Node.js o os.environ en Python). Esto separa las credenciales y configuraciones sensibles del código fuente, mejorando la seguridad y portabilidad.

Desarrollo del Frontend – Versión Inicial

Se desarrolló una interfaz web integrada con el backend bajo un enfoque de programación orientada a objetos, estructurada en:

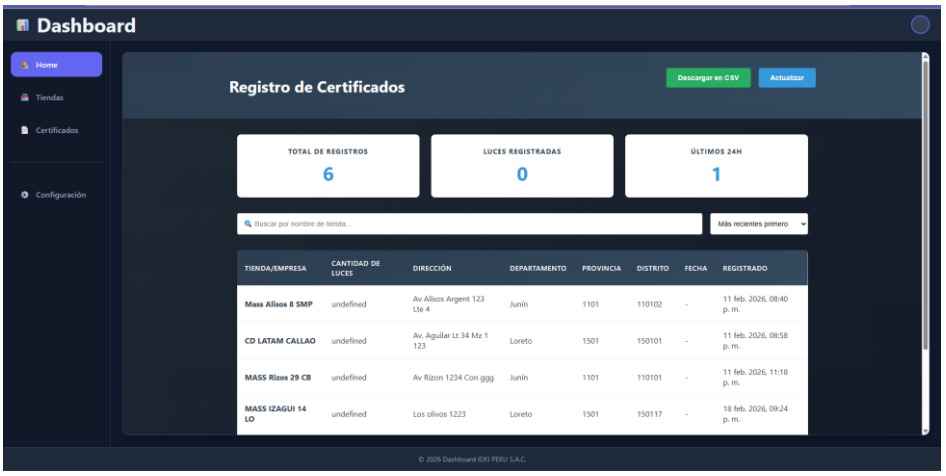
Figura 5

Arquitectura modular Backend.

Frontend desarrollado con HTML, CSS y JavaScript, servido directamente desde el backend como un archivo estático. La navegación entre rutas se implementa mediante **iframes**, lo que permite una interacción sencilla con las distintas vistas.

Figura 6

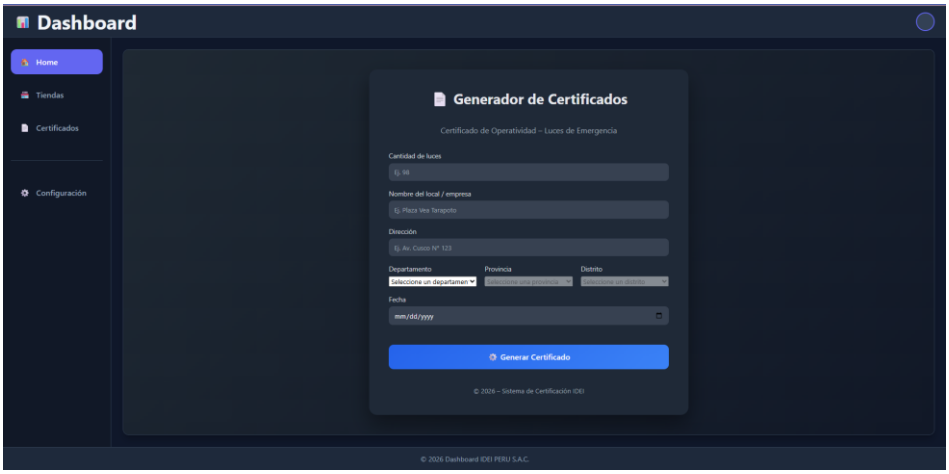
Dashboard.



Centraliza la interacción con el backend. Este panel ofrece una interfaz gráfica para visualizar datos, gestionar recursos y monitorear procesos internos. Su diseño combina simplicidad y funcionalidad, permitiendo al usuario navegar entre diferentes secciones de manera intuitiva.

Figura 7

Formulario de certificados.



Formulario dinámico que permite la creación de tiendas y certificados de manera flexible. Este formulario está diseñado para capturar información clave, como la cantidad de luces, fecha de emisión, nombre de la tienda, entre otros.

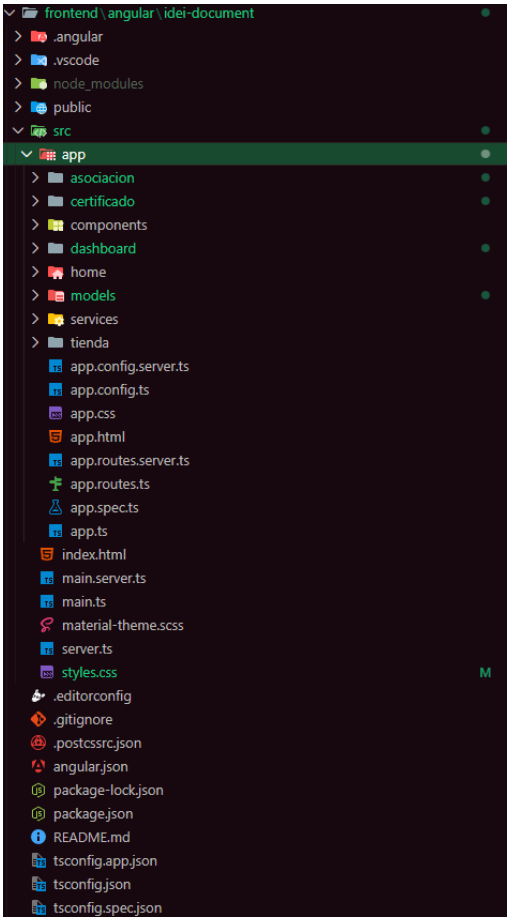
Migración y Mejora con Angular

Angular es un Framework de JavaScript de código abierto escrito en TypeScript. Su objetivo principal es desarrollar aplicaciones de una sola página.

Posteriormente, el sistema fue reestructurado utilizando Angular y TypeScript para mejorar la Escalabilidad, organización del código, experiencia de usuario (UX/UI) y adicionalmente se incorporó la librería Angular Material para mejorar los componentes visuales.

Figura 8

Arquitectura modular Frontend Angular.



Frontend desarrollado Angular, complementado con Angular Material, CSS y TypeScript, lo que permite construir una interfaz moderna, dinámica y altamente mantenible. La arquitectura modular se organiza en componentes y servicios

reutilizables, siguiendo principios de separación de responsabilidades y escalabilidad.

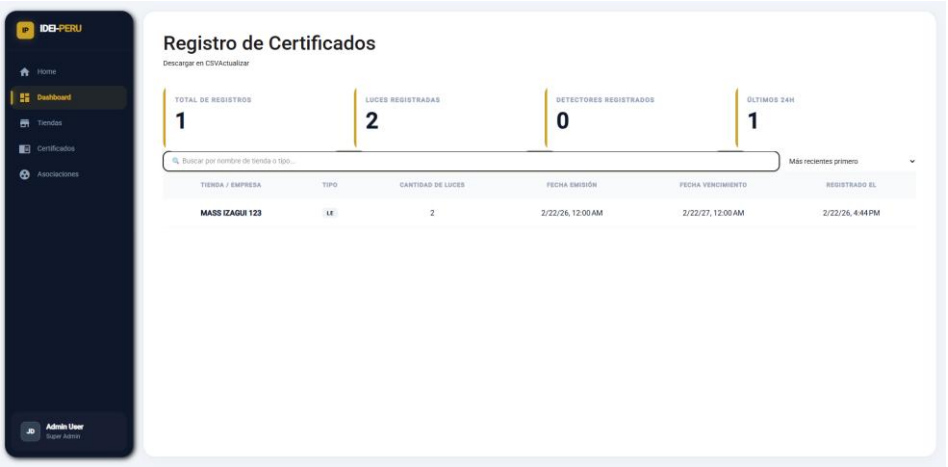
Figura 9

Home page.

El home page constituye el punto de entrada principal del sistema y centraliza la interacción con las diferentes rutas definidas en el frontend. Su diseño modular permite acceder de manera intuitiva a los apartados más relevantes de la aplicación: Dashboard, Certificados, Tiendas y Asociaciones.

Figura 10

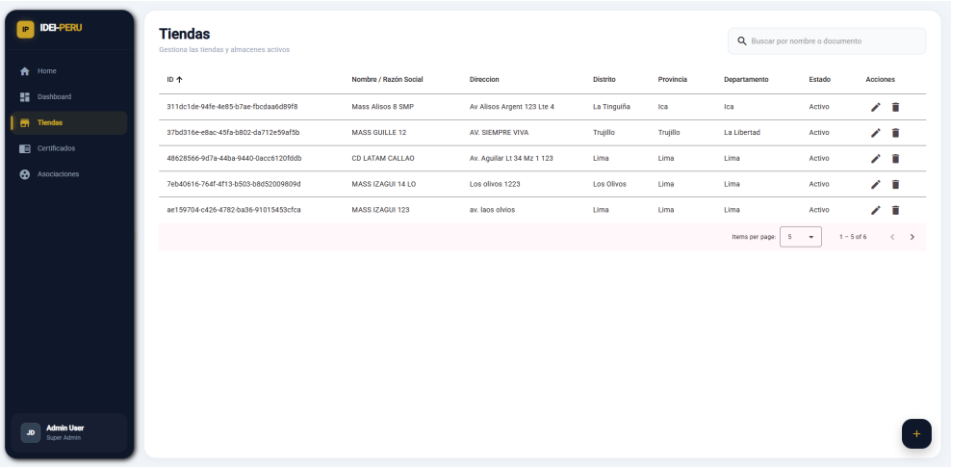
Dashboard page.



Centraliza la interacción con el backend. Este panel ofrece una interfaz gráfica para visualizar datos, gestionar recursos y monitorear procesos internos.

Figura 11

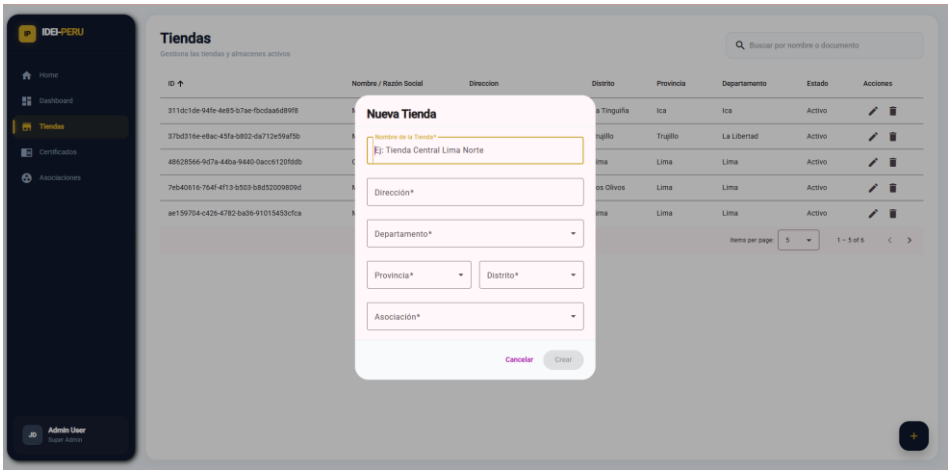
Tiendas page.



Listado de tiendas creadas por el usuario final.

Figura 12

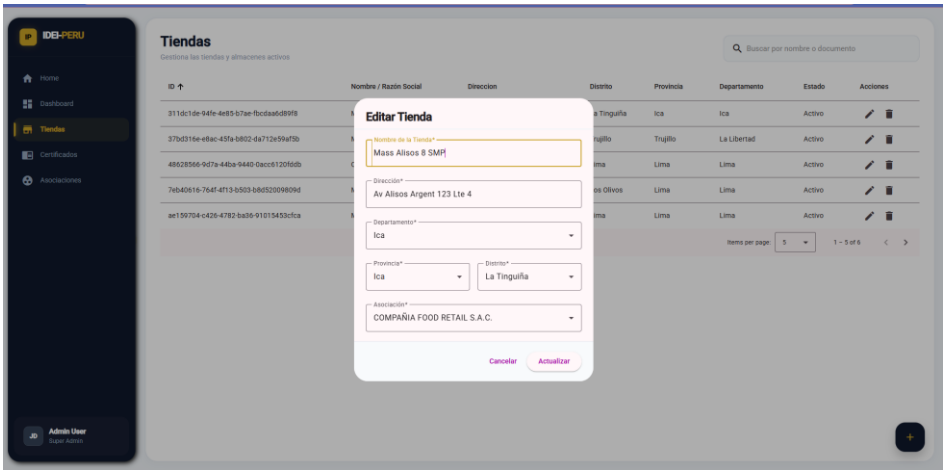
Tiendas modal crear.



Modal para agregar nuevas tiendas.

Figura 13

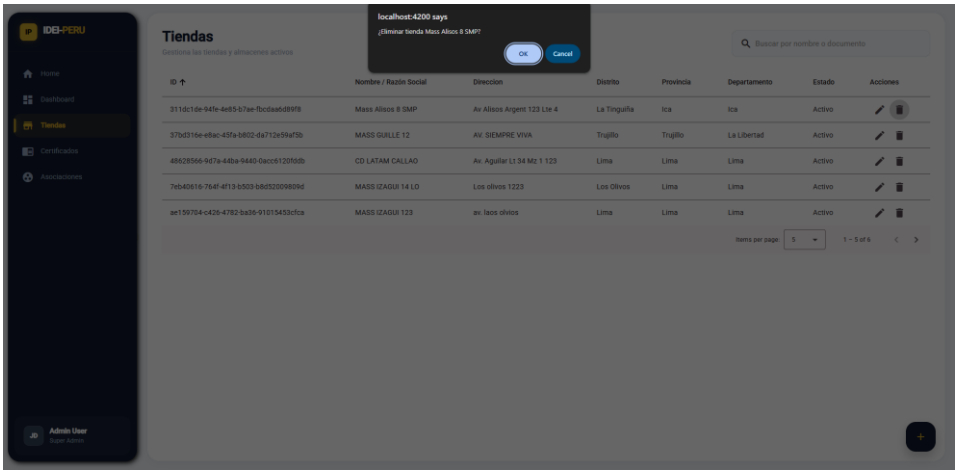
Tiendas modal editar.



Modal para editar las tiendas existentes

Figura 14

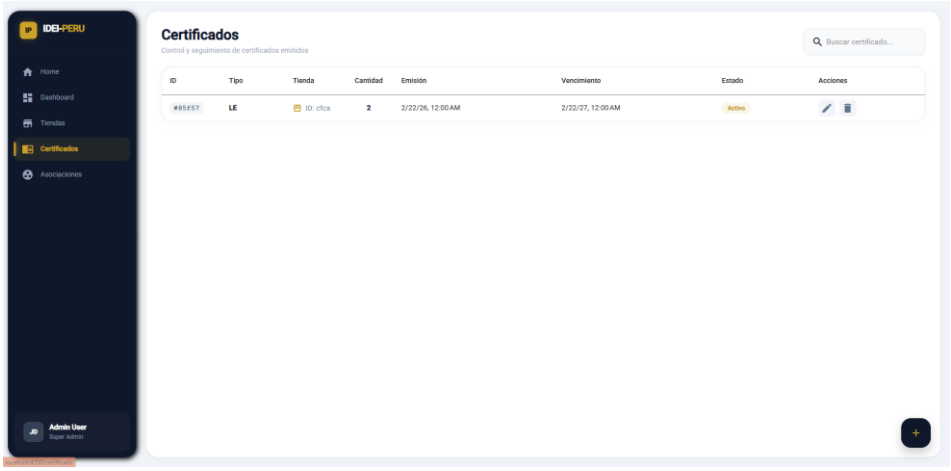
Tiendas alerta de eliminación.



Alerta de eliminación por tienda.

Figura 15

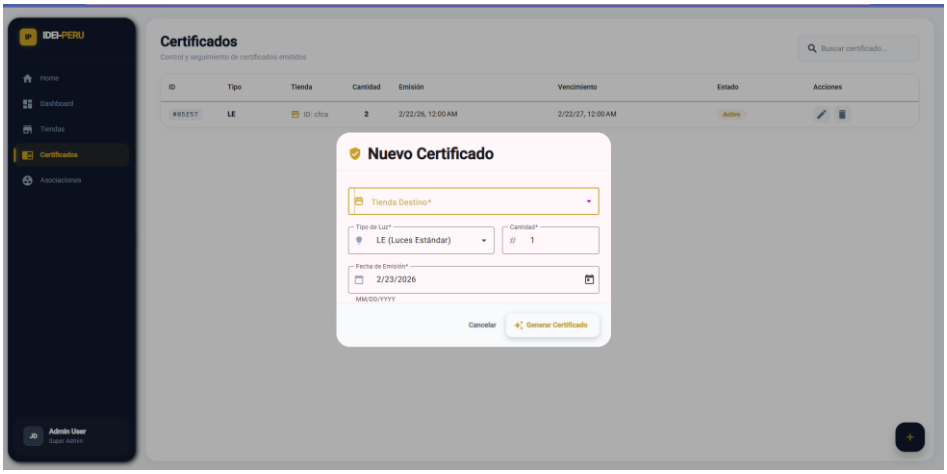
Certificados Page.



Listado de certificados creados por el usuario final.

Figura 16

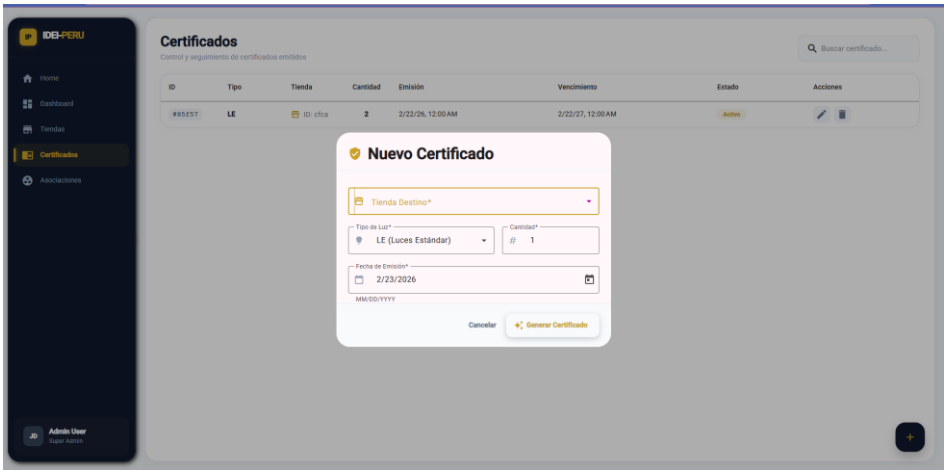
Certificados Modal Crear.



Modal para agregar nuevos certificados.

Figura 17

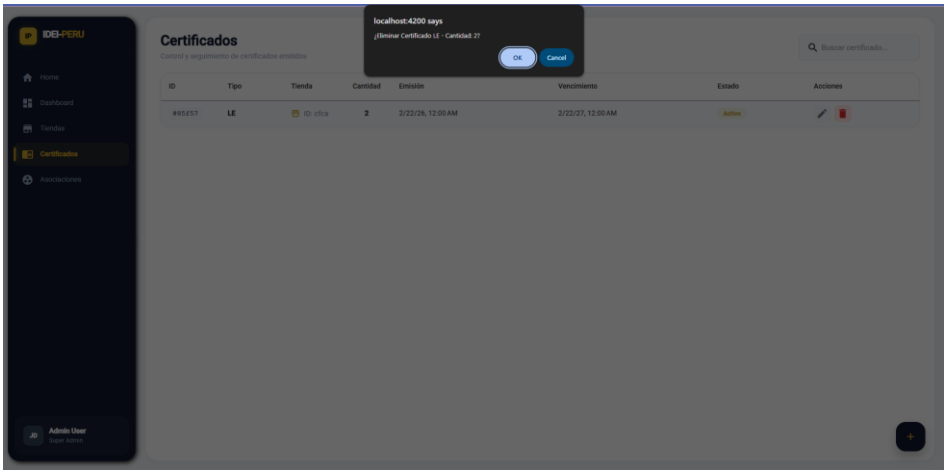
Certificados Modal Editar.



Modal para editar los certificados existentes

Figura 18

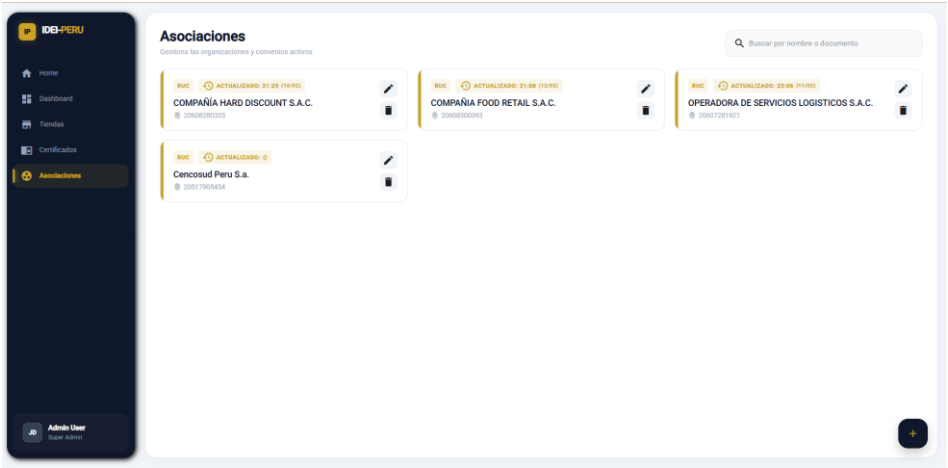
Certificado alerta de eliminación.



Alerta de eliminación por certificado.

Figura 19

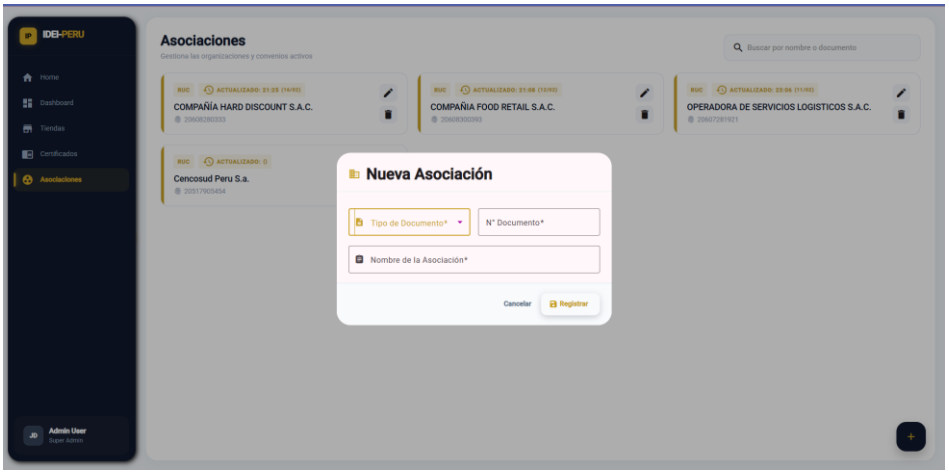
Asociaciones Page.



Listado de asociaciones agregadas por el usuario final.

Figura 20

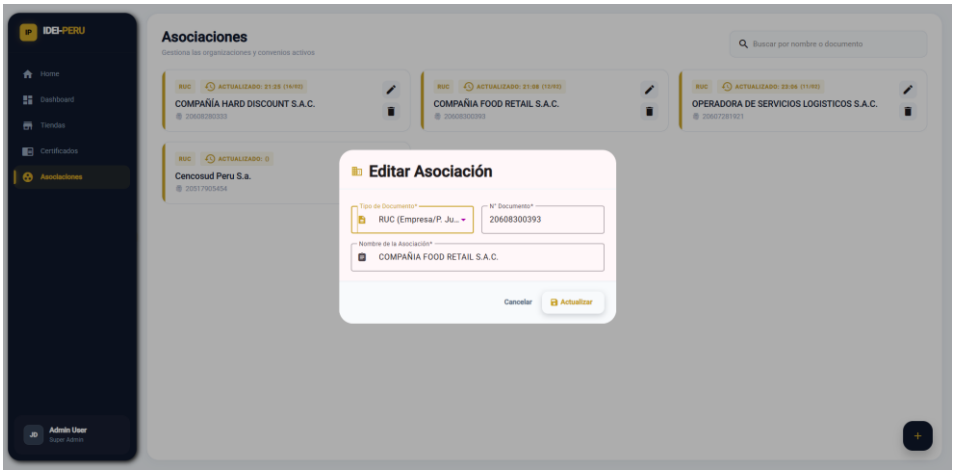
Asociaciones Modal Crear.



Modal para agregar nuevas asociaciones.

Figura 21

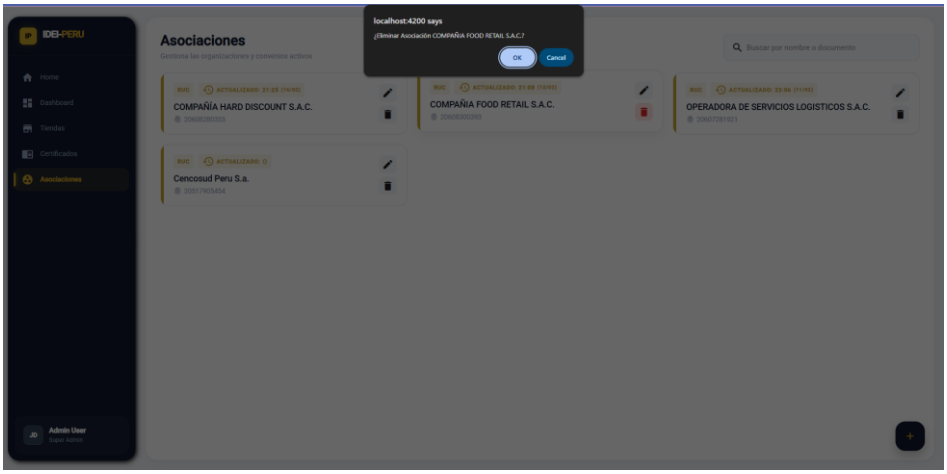
Asociaciones Modal Editar.



Modal para editar las asociaciones existentes

Figura 22

Asociaciones alerta de eliminación.



Alerta de eliminación por asociación.

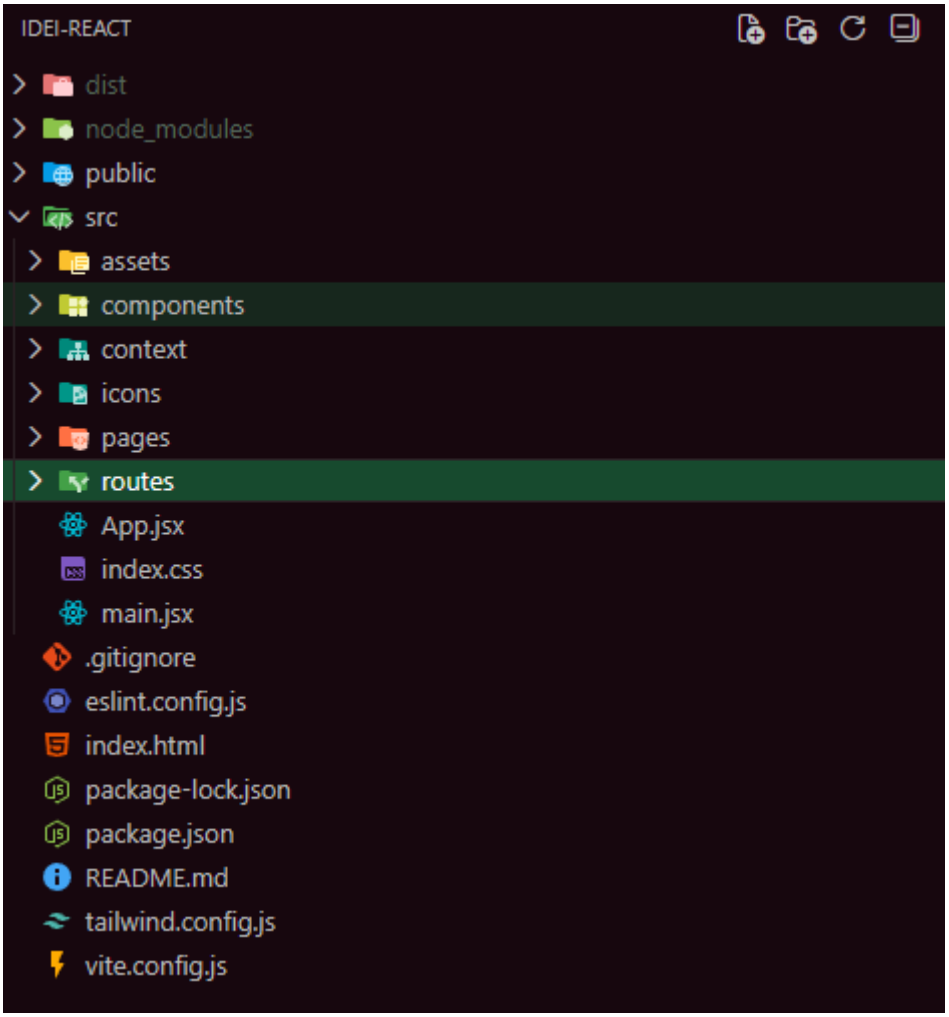
Desarrollo del Landing Page – React

React es una de las bibliotecas JavaScript más populares entre los desarrolladores Front End, programadores y testers de software. Te permite crear interfaces de usuario interactivas, ahorrando tiempo y reduciendo los costos de desarrollo.

El objetivo: mejorar la presencia digital de la empresa, se desarrolló un Landing Page utilizando React.

Figura 23

Arquitectura modular Frontend React landing page.



Frontend desarrollado React, tipo SPA (Single Page Application), donde el contenido dinámico se gestiona mediante React Router. Adicionalmente se

implementó un sistema de tema dinámico (theme schema) utilizando useContext y useMemo para optimización de renderizado.

Figura 23

Hero Lanfing page.

Espacio introductorio e informativo dentro del sistema. Su propósito es presentar de manera clara y visual los trabajos realizados en materia de seguridad y prevención en almacenes y tiendas.

Figura 24

Apartado Luces de emergencia.

Espacio informativo acerca del producto luces de emergencia.

Figura 25

Slider y especificaciones técnicas de las luces de emergencia

Figura 26

Apartado Nosotros.

Espacio informativo dentro del sistema, diseñado para presentar la identidad institucional y los valores que sustentan el proyecto.

Conclusiones:

El desarrollo del sistema permitió transformar un proceso manual en un sistema automatizado eficiente y estructurado. La adopción de tecnologías modernas como Angular y React permitió mejorar la experiencia del usuario y sentar bases sólidas para futuras mejoras.

El uso de plantillas HTML para generación de PDF mediante Puppeteer resultó ser una solución eficiente y escalable frente a la conversión directa de documentos ofimáticos.

El proyecto demuestra la aplicación práctica de:

- Programación orientada a objetos
- Arquitectura modular
- Desarrollo frontend y backend
- Optimización de procesos empresariale

Cuadro comparativo

Criterio	JavaScript Vanilla	React	Angular
Tipo de proyecto ideal	Páginas simples, landing pages básicas, formularios pequeños	Aplicaciones dinámicas, SPA, dashboards, e-commerce	Sistemas empresariales complejos, ERP, CRM, plataformas grandes
Complejidad del proyecto	Baja	Media	Alta
Curva de aprendizaje	Baja	Media	Alta
Estructura del proyecto	No estructurada (manual)	Modular por componentes	Arquitectura completa integrada (MVC/MVVM)
Escalabilidad	Limitada	Alta	Muy alta
Mantenimiento a largo plazo	Difícil si crece mucho	Moderado	Más estructurado y controlado
Tiempo de desarrollo inicial	Muy rápido	Rápido	Más lento al inicio
Inversión a corto plazo	Baja	Media	Media-Alta
Inversión a largo plazo	Puede aumentar por desorganización	Sostenible	Más rentable en proyectos grandes
Gestión de estado	Manual	Hooks / Context / Redux	Servicios, RxJS
Dependencias externas	Mínimas	Flexibles	Muchas integradas
Orientado a empresa grande	No recomendado	Sí	Altamente recomendado
SEO	Bueno si es estático	Bueno con SSR (Next.js)	Bueno con SSR (Angular Universal)
Control total del código	Alto	Alto	Alto pero más estructurado
Ideal para equipo grande	No	Sí	Sí (mejor estructuración)

Bibliografia:

Angular. (2024). *Angular documentation*. Google.

https://angular.io/docs

Angular Material. (2024). *Angular Material documentation*. Google.

https://material.angular.io

ConvertAPI. (2024). *ConvertAPI JavaScript library documentation*.

https://www.convertapi.com

CORS. (2024). *Cross-Origin Resource Sharing (CORS)*. Mozilla Developer Network.

https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

Dotenv. (2024). *Dotenv documentation*.

https://github.com/motdotla/dotenv

Express.js. (2024). *Express - Node.js web application framework*.

https://expressjs.com

HTML Living Standard. (2024). *HTML specification*. WHATWG.

https://html.spec.whatwg.org

Material Design. (2024). *Material Design guidelines*. Google.

https://m3.material.io

Mozilla Developer Network (MDN). (2024). *Cascading Style Sheets (CSS)*.

https://developer.mozilla.org/en-US/docs/Web/CSS

Mozilla Developer Network (MDN). (2024). *JavaScript reference*.

https://developer.mozilla.org/en-US/docs/Web/JavaScript

Node.js. (2024). *Node.js documentation*.

https://nodejs.org

Puppeteer. (2024). *Puppeteer documentation*. Google Chrome Developers. https://pptr.dev

React. (2024). *React documentation*. Meta. https://react.dev

React Router. (2024). *React Router documentation*. https://reactrouter.com

TypeScript. (2024). *TypeScript documentation*. Microsoft. https://www.typescriptlang.org/docs